Material de preparación para la clase 1: Fundamentos de Java

Introducción a la Programación Orientada a Objetos

Indicaciones: copia todo este material a mano en hojas cuadriculadas y tráelo a la clase 1. No se acepta impreso ni fotocopiado. Pon tu nombre en cada hoja. Al copiar, lee cada ejemplo y trata de entender qué hace; en clase se harán preguntas sobre este contenido.

Hoja 1: Tipos de datos

Un **tipo de dato** indica qué clase de valor puede guardar una variable y cuánta memoria ocupa. En Java toda variable debe declararse con su tipo antes de usarse.

Tipo	Para qué sirve	Ejemplo de declaración
int	Números enteros, positivos o negativos, sin decimales	int edad = 19;
double	Números con decimales	double nota = 8.5;
char	Un solo carácter, escrito entre comillas simples	char letra = 'A';
boolean	Solo puede valer true (verdadero) o false (falso)	boolean aprobado = true;
String	Texto: una o varias letras, escrito entre comillas dobles	String nombre = "Ana";

Notas:

- int, double, char y boolean son **tipos primitivos**: guardan directamente el valor.
- String **no es un tipo primitivo**: es una clase de Java. Por eso empieza con mayúscula.
- Otros tipos primitivos que existen: byte, short y long (enteros de distinto tamaño) y float (decimales con menos precisión que double).
- Los nombres de las variables se escriben en camelCase: empiezan con minúscula y cada palabra nueva va con mayúscula, por ejemplo notaFinal o esEstudiante.

Hoja 2: Estructura de un programa en Java

El programa más pequeño en Java:

```
// Mi primer programa
public class Saludo {
    public static void main(String[] args) {
        System.out.println("Hola");
    }
}
```

Parte	Qué significa
// Mi primer programa	Comentario de una línea. Java lo ignora; sirve para explicar el código.
/* ... */	Comentario de varias líneas. Todo lo que está entre /* y */ se ignora.
public class Saludo	Declara una clase llamada Saludo. El archivo debe llamarse igual:

Parte	Qué significa
	Saludo.java. El nombre de la clase empieza con mayúscula.
public static void main(String[] args)	Es el método principal. El programa empieza a ejecutarse aquí. Se escribe siempre igual.
{ }	Las llaves marcan dónde empieza y dónde termina un bloque (la clase, el método, un if, un for). Cada llave que se abre debe cerrarse.
;	El punto y coma marca el final de cada instrucción.
System.out.println("Hola");	Muestra un texto en pantalla y luego salta a la línea siguiente.
System.out.print("Hola");	Muestra un texto en pantalla pero no salta de línea.

Regla de sangría: todo lo que está dentro de un bloque se escribe con cuatro espacios más hacia la derecha. Java no lo exige, pero sin sangría el código es muy difícil de leer.

Hoja 3: Operadores

Aritméticos

Operador	Significado	Ejemplo	Resultado
+	Suma	7 + 2	9
-	Resta	7 - 2	5
*	Multiplicación	7 * 2	14
/	División	7 / 2	3 (si ambos son int)
/	División	7.0 / 2	3.5 (si uno es double)
%	Residuo de la división	7 % 2	1

Relacionales (comparan y dan true o false)

Operador	Significado	Ejemplo	Resultado
==	Igual a	5 == 5	true
!=	Diferente de	5 != 3	true
>	Mayor que	5 > 3	true
<	Menor que	5 < 3	false
>=	Mayor o igual que	5 >= 5	true
<=	Menor o igual que	4 <= 3	false

Lógicos (unen condiciones)

Operador	Significado	Ejemplo	Resultado
&&	Y: las dos condiciones deben ser verdaderas	(5 > 3) && (2 > 1)	true
||	O: basta con que una sea verdadera	(5 > 8) || (2 > 1)	true
!	No: invierte el valor	!(5 > 3)	false

Asignación

Operador	Significado	Ejemplo
=	Guarda un valor en una variable	int x = 10;

Diferencia entre = y ==: el signo = guarda un valor en una variable (asignación). El doble signo == pregunta si dos valores son iguales (comparación) y da true o false.

Hoja 4: Estructuras de control

if: decide si se ejecuta un bloque

Se ejecuta el bloque solo si la condición es verdadera.

```
if (nota >= 7) {
    System.out.println("Aprobado");
}
```

if / else: elige entre dos caminos

Si la condición es verdadera se ejecuta el primer bloque; si no, el segundo.

```
if (nota >= 7) {
    System.out.println("Aprobado");
} else {
    System.out.println("Reprobado");
}
```

if / else if / else: elige entre varios caminos

Revisa las condiciones en orden y ejecuta solo la primera que sea verdadera.

```
if (nota >= 7) {
    System.out.println("Aprobado");
} else if (nota >= 5) {
    System.out.println("Supletorio");
} else {
    System.out.println("Reprobado");
}
```

for: repite un número conocido de veces

Tiene tres partes separadas por punto y coma:

1. Inicio: se ejecuta una sola vez al empezar (int i = 1).
2. Condición: se revisa antes de cada vuelta; si es falsa, el ciclo termina (i <= 5).
3. Incremento: se ejecuta al final de cada vuelta (i++).

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
// Muestra 1, 2, 3, 4 y 5, cada uno en una línea
```

while: repite mientras una condición sea verdadera

Se usa cuando no se sabe de antemano cuántas vueltas habrá. Dentro del bloque algo debe cambiar la condición; si no, el ciclo nunca termina.

```
int saldo = 200;
while (saldo < 500) {
    saldo = saldo + 50;
}
System.out.println(saldo); // Muestra 500
```

Diferencia entre for y while: el for se usa cuando se conoce cuántas veces se repite; el while, cuando la repetición depende de una condición que cambia durante el programa.

Hoja 5: Glosario

Término	Definición
Variable	Espacio de memoria con un nombre, donde se guarda un valor que puede cambiar.
Tipo de dato	Indica qué clase de valor guarda una variable: entero, decimal, texto, etc.
Declaración	Crear una variable indicando su tipo y su nombre. Ejemplo: int edad;
Asignación	Guardar un valor en una variable. Ejemplo: edad = 19;
Inicialización	Declarar y asignar en la misma línea. Ejemplo: int edad = 19;
Sentencia	Una instrucción completa del programa; termina en punto y coma.
Bloque	Grupo de sentencias encerradas entre llaves { }.
Condición	Expresión que da como resultado true o false.
Ciclo	Estructura que repite un bloque varias veces (for, while).
Compilar	Revisar el código y traducirlo a un formato que la computadora puede ejecutar. Si hay errores de escritura, no compila.
Ejecutar	Poner en marcha el programa ya compilado.
Error de compilación	Error de escritura que impide compilar, por ejemplo olvidar un punto y coma.
Error de ejecución	Error que aparece mientras el programa corre, por ejemplo convertir "abc" a número.
Prueba de escritorio	Seguir el programa a mano, línea por línea, anotando en una tabla el valor de cada variable para saber qué hace y qué muestra.

Hoja 6: Atajos y trucos

Incremento y decremento

Atajo	Equivale a	Ejemplo
b++;	b = b + 1;	si b vale 4, queda en 5
b--;	b = b - 1;	si b vale 4, queda en 3

Diferencia entre b++ y ++b: solo importa cuando están dentro de otra expresión.

```
int a = 5;
int c = a++;   // c vale 5 y a vale 6: primero usa el valor, después suma
```

```
int x = 5;
int y = ++x;   // y vale 6 y x vale 6: primero suma, después usa el valor
```

Asignación compuesta

Atajo	Equivale a	Si x vale 10, queda en
x += 3;	x = x + 3;	13
x -= 3;	x = x - 3;	7
x *= 2;	x = x * 2;	20
x /= 2;	x = x / 2;	5
x %= 3;	x = x % 3;	1

Acumular dentro de un for

```
int suma = 0;
for (int i = 1; i <= 10; i++) {
    suma += i;
}
System.out.println(suma); // Muestra 55
```

División y residuo

- 7 / 2 da 3 cuando los dos son int: se pierde el decimal.
- 7 / 2.0 da 3.5: basta con que uno sea double.
- 7 % 2 da 1, el residuo.
- n % 2 == 0 es true si n es par.

Texto

- "Nota: " + 8 da "Nota: 8": el + con un texto une (concatena).
- "1" + 2 + 3 da "123", pero 1 + 2 + "3" da "33": Java evalúa de izquierda a derecha.
- \n dentro de un texto hace un salto de línea: "Hola\nAna" se muestra en dos líneas.

Comparar

- Los números se comparan con ==.
- Los textos se comparan con equals: nombre.equals("Ana"). Con == puede fallar.
- !(a > b) es lo mismo que a <= b.

Errores típicos

- if (x = 5) es un error: para comparar se usa ==.
- Un ; después de if (...) o de for (...) deja el bloque vacío y el programa hace algo distinto a lo esperado.
- No se puede declarar dos veces la misma variable en el mismo bloque.
- No se puede usar una variable a la que nunca se le asignó un valor.

Hoja 7: Conversiones entre tipos

Convertir es pasar el valor de una variable de un tipo a otro. En cada ejemplo se declara primero la variable de origen con su valor y luego la variable nueva donde se guarda el resultado. El comentario // indica el valor que queda en la variable nueva.

De texto (String) a número

```
String textoEdad = "25";
int edad = Integer.parseInt(textoEdad);           // edad = 25

String textoNota = "8.75";
double nota = Double.parseDouble(textoNota);      // nota = 8.75

String textoActivo = "true";
boolean activo = Boolean.parseBoolean(textoActivo);
// activo = true

String saludo = "Hola";
char inicial = saludo.charAt(0);                  // inicial = 'H'
```

Variable de origen	Valor	Variable nueva	Tipo nuevo	Valor nuevo
textoEdad	"25"	edad	int	25
textoNota	"8.75"	nota	double	8.75
textoActivo	"true"	activo	boolean	true
saludo	"Hola"	inicial	char	'H'

De número a texto (String)

```
int puntos = 25;
String textoPuntos1 = String.valueOf(puntos);     // textoPuntos1 = "25"
String textoPuntos2 = Integer.toString(puntos);   // textoPuntos2 = "25"
String textoPuntos3 = "" + puntos;                // textoPuntos3 = "25"

double precio = 8.5;
String textoPrecio = String.valueOf(precio);      // textoPrecio = "8.5"

char letra = 'A';
String textoLetra = String.valueOf(letra);        // textoLetra = "A"

boolean aprobado = true;
String textoAprobado = String.valueOf(aprobado);  // textoAprobado = "true"
```

Variable de origen	Valor	Variable nueva	Tipo nuevo	Valor nuevo
puntos	25	textoPuntos1	String	"25"
precio	8.5	textoPrecio	String	"8.5"
letra	'A'	textoLetra	String	"A"
aprobado	true	textoAprobado	String	"true"

Entre números (int y double)

```
int horas = 7;
double horasDecimal = horas;                        // horasDecimal = 7.0

double promedio = 8.9;
int promedioEntero = (int) promedio;                // promedioEntero = 8

double calificacion = 8.6;
long redondeo = Math.round(calificacion);           // redondeo = 9

double otraCalificacion = 8.4;
int redondeoEntero = (int) Math.round(otraCalificacion); // redondeoEntero =
8

int total = 7;
int partes = 2;
int divisionEntera = total / partes;                // divisionEntera = 3
double division = (double) total / partes;          // division = 3.5
```

Variable de origen	Valor	Variable nueva	Tipo nuevo	Valor nuevo
horas	7	horasDecimal	double	7.0
promedio	8.9	promedioEntero	int	8
calificacion	8.6	redondeo	long	9
otraCalificacion	8.4	redondeoEntero	int	8
total y partes	7 y 2	divisionEntera	int	3
total y partes	7 y 2	division	double	3.5

Nota: "" + puntos es un atajo para pasar un número a texto. De int a double Java convierte solo. De double a int hay que escribir (int) delante: corta el decimal, no redondea. Math.round devuelve un long; para guardarlo en un int también se usa (int).

Entre char y número

```
char letraA = 'A';
int codigo = letraA;                                // codigo = 65

int codigoB = 66;
char letraB = (char) codigoB;                       // letraB = 'B'

char digito = '7';
int numero = digito - '0';                          // numero = 7
```

Variable de origen	Valor	Variable nueva	Tipo nuevo	Valor nuevo
letraA	'A'	codigo	int	65
codigoB	66	letraB	char	'B'
digito	'7'	numero	int	7

Nota: cada carácter tiene un código numérico. 'A' es 65, 'B' es 66, 'a' es 97. Por eso '7' - '0' da 7.

Errores típicos

```
String palabra = "abc";
int n1 = Integer.parseInt(palabra);     // ERROR al ejecutar

String decimal = "8.5";
int n2 = Integer.parseInt(decimal);     // ERROR al ejecutar

String conEspacio = " 25";
int n3 = Integer.parseInt(conEspacio.trim()); // n3 = 25

String cinco = "5";
String unido = cinco + 3;               // unido = "53"
int suma = Integer.parseInt(cinco) + 3; // suma = 8
```

Línea	Qué pasa
n1	NumberFormatException: "abc" no es un número
n2	Falla porque "8.5" tiene decimales; se convierte con Double.parseDouble
n3	trim() quita el espacio; sin trim() daría error
unido	Texto más número se concatena: "53"
suma	Primero se convierte "5" a 5 y luego se suma: 8

Ejercicio de repaso (resuelto)

```
String doce = "12";
int r1 = Integer.parseInt(doce) + 3;  // r1 = 15
String r2 = doce + 3;                 // r2 = "123"

double casi = 9.99;
int r3 = (int) casi;                  // r3 = 9

double medio = 9.5;
long r4 = Math.round(medio);          // r4 = 10

char a = 'a';
char r5 = (char) (a + 1);             // r5 = 'b'
```

Variable	Tipo	Valor	Por qué
r1	int	15	Convierte "12" a 12 y luego suma 3
r2	String	"123"	Un texto más un número se concatena
r3	int	9	El casting corta el decimal
r4	long	10	Math.round redondea hacia arriba desde .5
r5	char	'b'	Suma 1 al código de 'a' (97) y lo vuelve letra